

ASSIGNMENT 2

Each exercise is independent. Don't forget to re-export your session link after completing each exercise. You should have a link per exercise. You can email me your solutions' links at nicolas.gry@inria.fr

Exercise 1 — Playlist analyser

The exercise should be completed using this link: <https://godbolt.org/z/T6Eqq6z8T>.

The goal of this exercise is to write a program that analyses a short playlist of songs represented as integer scores (0–100). The program will tell the quality of each song – ‘Poor’, ‘Good’ or ‘Excellent’ – depending on its score. It will also print the rating of the best score. The songs scores will be supplied entirely as arguments to `main`.

Invocation:

```
./playlist 72 95 40 88 55
```

Each argument after the program name is the score of one song.

Step 1 — Categorise each score.

Given a score, you have to rate a song — i.e the song can be ‘Poor’, ‘Good’ or ‘Excellent’. The code in the provided link already defines the `enum class Rating` for that. Complete the following function:

```
Rating rate(int score) {
    Rating the_rate;
    if(score < 50){
        the_rate = Rating::Poor;
    }
    // Step 1: complete me

    return the_rate;
}
```

The function should match a rating to the score it is given. The case for a ‘poor’ score has been written as an example. The matching between score and rating follow these criteria:

Condition	Rating
score < 50	Poor
50 <= score < 80	Good
otherwise	Excellent

Step 2 — Categorise each score.

We need to be able to output the rating. To that end, complete the function `print_rating` so that it prints "poor", "good", or "excellent" depending on the rating it is given. The case for 'poor' is already written as an example.

```
void print_rating(Rating rate){
    switch(rate){
        case Rating::Poor:
            std::cout << "poor" << std::endl;
            break;
        // Step 2: complete me
    }
}
```

Step 3 — Loop and report.

In `main`, a for-loop has been written. Currently, it loops over the scores provided to the program and prints them:

```
1 int highest_score = 0;
2 for(auto arg = 1; arg < argc; ++arg){
3     int score = atoi(argv[arg]);
4
5     std::cout << "score " << score << ":";
6     std::cout << std::endl;
7     // Step 3: complete me
8 }
```

On each iteration, the variable `score` contains the next score in the list of scores provided by the user.

For each score, print its rating by calling `print_rating`. Track the highest score value in the variable `highest_score`.

Step 4 — Summary.

We have managed to print the rate for each score. We now need to print the rate of the best score.

Call `print_rating(rate(highest_score))` to print the rating of the best song on a line:

```
best rating: <rating>
```

Expected output for `./playlist 72 95 40 88 55`:

```
score 72: good
score 95: excellent
score 40: poor
score 88: excellent
score 55: good
best rating: excellent
```

Exercise 2 — Image colour corrector

This exercise should be completed using this link: <https://godbolt.org/z/7WbeKx9r9>.

The goal of this exercise is to write a program that applies a correction to a colored pixel. A pixel can be represented by three integers: the red value, the green value and the blue value – we talk about RGB values. They are always comprised between 0 and 255 (inclusive). Your program will take four inputs: the RGB values and an offset to add to each value.

Invocation:

```
./correct <r> <g> <b> <offset>
```

Step 1 — Global pixel state.

We start by storing the values provided by the user in our program. Three global variables `int red`, `int green`, `int blue` have already been declared and initialized to 0.

Complete the function `set_pixel` so that it stores the provided argument values in the variables `red`, `green` and `blue`. The red value has already been taken care of as an example.

```
void set_pixel(int rv, int gv, int bv){
    red = rv;
    // Step 1: complete me
}
```

Step 2 — Clamped offset.

The code already defines the function `clamp` which, given an integer, returns 0 if the integer is below 0; 255 if it is above 255; or the given integer otherwise.

```
1 int clamp(int value) {
2     if(value < 0) { return 0; }
3     if(value > 255) { return 255; }
4     return value;
5 }
```

We need a function that can add two values together and bring the result in the range [0, 255]. The function `apply` does that:

```
int apply(const int channel, const int offset) {  
    return clamp(channel + offset);  
}
```

We would like however a function that modifies its given argument in place. Complete the following function:

```
1 void apply(int& result, int channel, int offset){  
2     result = ...;  
3 }
```

so that `result` contains the same value that `apply(channel, offset)` would return.

Step 3 — Putting it all together

Let's put this all together. In the main function, the RGB values provided by the user are stored in the variables `r`, `g` and `b` and the offset is stored in `offset`.

1. Use `set_pixel` to store the value of `r` in `red`, `g` in `green` and `b` in `blue`.
2. Call the **reference** overload of `apply` with the following combinations of arguments:
 - `red, r, offset`
 - `green, g, offset`
 - `blue, b, offset`

3. Print the corrected pixel values:

```
corrected: <r> <g> <b>
```

Expected output for `./correct 200 100 30 80`:

```
original: 200 100 30  
corrected: 255 180 110
```

Expected output for `./correct 100 120 80 20`:

```
original: 100 120 80  
corrected: 120 140 100
```